

Phase 3 Agent Engineering: Project Assignment

Full-Stack MERN + Agentic AI | Production-Level Build

What this phase is really about: Phase 2 taught you to call an LLM and ground it with RAG. That produces a system that answers questions. Agent engineering produces a system that **takes actions** - it decides what to do, does it, checks whether it worked, and knows when to stop and ask a human.

That shift changes the engineering problem completely. A chatbot that returns a wrong answer wastes a user's time. An agent that decides wrongly issues a refund, sends an email, or deletes a record. Every requirement in this assignment exists because of that difference.

Skill mapping - every Phase 3 topic is forced by this build, not bolted on:

Phase 3 Topic	Where it becomes unavoidable
AI Agent Fundamentals	Build 1 - the core agent loop
Tool Usage	Build 1 - tools with real side effects
Planning	Build 2 - multi-step tickets needing a plan before action
Reflection	Build 2 - the agent grading its own output before returning it
Human-in-the-Loop	Build 2 - approval gates on irreversible actions
Workflow vs Autonomous Agents	Build 2 - you must choose per task type and defend it
Multi-Agent Systems	Build 3 - specialist agents with a coordinator
Multi-Agent Research System	Build 3 - the research agent pattern
Voice AI with RAG	Build 3 - voice channel over the same agent core

Assignment Overview

One system, three builds. Each build is reviewed before the next starts.

Build	Focus	Duration
Build 1	Single agent, tool calling, streaming, safety rails	20–25 hrs
Build 2	Planning, reflection, HITL, workflow-vs-autonomous routing	25–30 hrs
Build 3	Multi-agent orchestration, research agent, voice channel	25–30 hrs

THE PROJECT

HelpDesk Copilot - An Agentic Customer Support Platform

The Domain

A SaaS company's support team handles tickets that require *doing things*, not just answering questions:

"My last invoice charged me twice - please refund the duplicate and tell me why it happened."

Answering this requires: looking up the customer, fetching their invoice history, identifying the duplicate charge, checking refund eligibility against policy, **issuing the refund** (irreversible, money moves), and explaining the root cause. That's retrieval, reasoning, a policy check, a side-effecting action, and a written explanation - in one request.

Why This Domain

Support is the ideal agent domain precisely because it contains the full risk spectrum in one system:

Action	Reversible?	Should the agent do it alone?
Look up order status	Yes (read-only)	Yes - fully autonomous
Draft a reply	Yes	Yes
Apply a \$10 account credit	Mostly	Yes, within policy limits

Issue a \$2,400 refund	No	No - human approval required
Delete a customer account	No	Never - always escalate

That gradient is exactly what makes human-in-the-loop, autonomy boundaries, and reflection real design requirements instead of textbook concepts.

BUILD 1 - Single Agent with Tool Calling

Difficulty

★★★★☆ | 20–25 Hours

Objective

Build a working agent loop: the LLM receives a request, decides which tools to call, calls them, observes results, and either continues or produces a final answer. Everything else in this project sits on top of this loop, so it must be correct and observable.

Tech Stack

- **Backend:** Node.js + Express
- **Frontend:** React or Next.js
- **DB:** MongoDB + Mongoose
- **Cache/State:** Redis
- **LLM:** any provider with native tool-calling support
- **Vector store:** ChromaDB (policy/knowledge retrieval)
- **Streaming:** SSE

Functional Requirements

The Agent Loop Implement the core cycle explicitly - do not use a framework's black-box `agent.run()`. You must own this loop:

1. Send conversation + tool definitions to the LLM
2. Model responds with either a tool call OR a final answer
3. If tool call → execute it → append the result to the conversation → repeat from 1
4. If final answer → stream to user, end

Hard requirements on the loop:

- **Max iteration cap** (e.g. 8) - an agent that loops forever burns money and hangs the request. Cap it and handle hitting the cap gracefully
- **Every iteration persisted** - the full trace (thought → tool call → arguments → result) must be inspectable afterwards
- **Token budget per request** - track cumulative tokens, abort cleanly if exceeded

Tools (minimum 7, with real behavior)

Tool	Type	Notes
<code>searchKnowledgeBase(query)</code>	Read	ChromaDB retrieval over help articles + policy docs
<code>getCustomer(email)</code>	Read	MongoDB
<code>getOrders(customerId, limit)</code>	Read	MongoDB
<code>getInvoices(customerId)</code>	Read	MongoDB
<code>checkRefundEligibility(orderId)</code>	Read + logic	Evaluates policy rules - the agent must not decide eligibility itself
<code>issueRefund(orderId, amount, reason)</code>	Write - irreversible	Money moves
<code>applyAccountCredit(customerId, amount)</code>	Write	Bounded by policy
<code>escalateToHuman(reason, priority)</code>	Write	Creates a human queue entry

Every tool must have: a JSON schema, **input validation before execution** (never trust LLM-generated arguments - the model will produce malformed or out-of-range values, this is expected), and structured error returns the agent can reason about (`{error: "customer_not_found", message: "..."} - not a thrown exception that kills the loop).`

Safety Rails (mandatory in Build 1)

- **Hard policy limits enforced in code, not in the prompt.** `issueRefund` must reject amounts above a threshold at the function level. A prompt saying "never refund over \$500" is a suggestion; a code check is a guarantee. This distinction is the single most important safety lesson in agent engineering
- **Confirmation requirement:** any write tool must be confirmed by the user in the UI before it executes (Build 2 replaces this with a proper approval workflow)

- **Grounding:** policy answers must come from retrieved documents, with citations. The agent must say it doesn't know rather than invent policy

Streaming (SSE) Stream more than tokens - stream the agent's *activity*:

```
event: thinking    → "Analyzing the request..."
event: tool_call   → { tool: "getInvoices", args: {...} }
event: tool_result → { tool: "getInvoices", summary: "Found 12 invoices" }
event: token       → partial final answer text
event: done        → { iterations: 4, tokensUsed: 3200, toolsCalled: [...] }
```

Frontend (React)

- **Chat interface** with live agent activity - the user sees each tool being called as it happens, not a spinner
- **Trace viewer:** expandable per-message view showing every iteration - reasoning, tool name, arguments, result, duration
- **Confirmation modal** for write actions, showing exactly what will happen before it happens
- **Cost/token counter** per conversation

Backend Requirements

- **Tool registry pattern** - adding a tool must be one registration, with zero changes to the agent loop
- **MongoDB:** conversations, messages, tool call records (with args, results, duration, success/failure), customers, orders, invoices, refunds
- **Redis:** conversation state, per-user rate limiting on agent requests (agent calls are expensive - an unbounded loop is a real cost incident)
- **Structured logging** with a correlation ID per agent run - one request may involve 8 LLM calls and 15 tool executions; that must be reconstructable from logs

Evaluation Criteria

Criteria	Marks
Agent Loop Correctness (iteration cap, error recovery, termination)	25
Tool Design (schemas, validation, structured errors)	20
Safety Rails (code-enforced limits, confirmation flow)	20
Streaming & Activity Visibility	15
Trace Persistence & Viewer	10

RAG Grounding with Citations

10

Total

100

Important Instructions

- No framework `agent.run()` abstraction - you must implement and own the loop. You can use frameworks in Build 3 once you've proven you understand what they're hiding
- A refund limit enforced only in the system prompt fails this build outright
- Demonstrate the failure paths: a tool returning an error, the iteration cap being hit, and a rejected out-of-policy refund

BUILD 2 - Planning, Reflection & Human-in-the-Loop

Difficulty

★★★★★ | 25–30 Hours

Objective

Build 1's agent is reactive - it decides one step at a time. Real tickets need a plan before acting, a quality check before responding, and a human before doing anything irreversible. This build adds all three, and forces the workflow-vs-autonomous decision.

New Requirements

Planning

- For any request classified as multi-step, the agent produces an **explicit plan before executing anything**: an ordered list of steps, each naming the tool it will use and its expected outcome
- The plan is **persisted and shown to the user** before execution begins
- **Replanning**: when a step fails or returns something unexpected, the agent revises the remaining plan rather than blindly continuing. Track and display plan revisions - a plan that changed twice mid-execution is a signal worth seeing
- Design decision to defend: which requests need a plan at all? Planning costs latency and tokens; "where's my order" doesn't need one. Where's your threshold, and how is it determined?

Reflection

- Before returning any final answer, a **reflection pass** evaluates the draft against explicit criteria: Does it answer the actual question? Is every claim grounded in a tool result or retrieved doc? Is the tone appropriate? Did it leak internal details?
- If reflection fails, the agent revises - **capped at 2 revision attempts** (unbounded self-revision is a cost and latency trap)
- **Both the reflection verdict and the original draft must be persisted** - you need to see what was rejected and why, and this becomes your quality dataset
- Design decision to defend: same model for reflection, or a cheaper/different one? There's a real tradeoff between cost, latency, and the known weakness of a model evaluating its own output

Human-in-the-Loop (this is the centerpiece)

- **Approval gates** on all irreversible actions, with tiering:

Condition	Behavior
Read-only tools	Autonomous
Writes under policy threshold	Autonomous, logged
Writes over threshold	Paused - agent queues an approval request and waits
Explicit escalation triggers (legal, security, account deletion)	Always human, agent never acts

- When paused, the agent run must **persist its full state** and resume correctly when approved - possibly minutes or hours later, possibly on a different server instance. This is the hardest engineering problem in Build 2: an agent loop that survives suspension
- **Agent operator queue (frontend)**: pending approvals showing the request, the agent's reasoning, the exact action proposed, and the customer context - with Approve / Reject / **Modify** (human edits the amount, agent proceeds with the edited value)
- **Rejection handling**: a rejected action must not crash the run - the agent must incorporate the rejection and respond to the customer appropriately

Workflow vs Autonomous Routing Not every ticket needs an agent. Implement a **classifier** that routes incoming requests:

Route	When	Example
-------	------	---------

Deterministic workflow (no LLM decisions)	High-volume, fully predictable	"Where is my order?" → fixed lookup → templated reply
Guided agent (agent with a constrained tool subset)	Known category, some judgment needed	Refund requests → only refund-related tools available
Autonomous agent (full toolset)	Novel or ambiguous	Anything not matching a known pattern

You must implement all three paths and **document why each category routes where it does**. Track the distribution - what percentage of traffic actually needs an autonomous agent? That number is the argument for this whole design, and it's usually surprisingly low.

Frontend additions

- Plan viewer: steps, live status, revisions highlighted
- Reflection panel: draft vs final, with the reflection verdict
- Approval queue with Approve/Reject/Modify
- Routing analytics: traffic split across the three paths

Backend Requirements

- **Durable agent state:** a paused run must survive a server restart. Redis for hot state, MongoDB for durability - and a clear reasoning for what lives where
- **Timeout on pending approvals:** what happens if no human responds in 24 hours? Design and defend it
- **Idempotent resumption:** approving twice (double-click, retry) must not execute the action twice

Evaluation Criteria

Criteria	Marks
Planning + Replanning Correctness	20
Reflection Loop (criteria, revision cap, persistence)	20
HITL: Pause, Persist, Resume Correctness	25
Approval Queue (incl. Modify + rejection handling)	10
Workflow vs Autonomous Routing (implemented + justified)	15
Idempotency of Resumption	10
Total	100

Important Instructions

- Demonstrate pause → **restart the server** → approve → agent resumes correctly. An in-memory pause fails this build
- Reflection that always passes is not reflection - show a case where it caught and fixed a bad draft
- If every request routes to the autonomous agent, the routing layer isn't doing its job

BUILD 3 - Multi-Agent System, Research Agent & Voice

Difficulty

★★★★★ | 25–30 Hours

Objective

Split one over-loaded agent into specialists with a coordinator, add a research agent that investigates open-ended questions, and expose the whole system over voice. This is where you learn what multi-agent actually buys you - and where it costs more than it's worth.

New Requirements

Multi-Agent Architecture

Agent	Owns	Tools
Coordinator	Classifies intent, delegates, assembles the final response	Delegation only - no domain tools
Billing Agent	Invoices, refunds, credits, payment issues	Billing tools + billing policy KB
Technical Agent	Product issues, troubleshooting, bug triage	Technical KB, system status, log lookup
Account Agent	Profile, subscription, plan changes	Account tools
Research Agent	Open-ended investigation (see below)	Read-only, multi-source

Requirements:

- Each agent has its **own system prompt, own tool subset, own KB collection** - the isolation is the point: a billing agent that can't touch account-deletion tools cannot misuse them
- **Coordinator handoff protocol:** a documented contract for what's passed to a specialist and what comes back. Design decision to defend: full conversation history, or a summarized brief? (context cost vs information loss - a real tradeoff)
- **Multi-agent requests:** *"My subscription didn't renew and I was charged anyway"* touches Account + Billing. The coordinator must sequence or parallelize them and merge results coherently
- **Loop prevention:** agents must not hand off back and forth indefinitely. Cap delegation depth and handle hitting the cap
- **Mandatory written analysis:** compare multi-agent against a single agent with all tools. Where does specialization genuinely help (prompt focus, tool isolation, independent evaluation), and where does it just add latency and handoff failure modes? Be honest - multi-agent is frequently over-applied, and recognizing that is a senior-level judgment

Research Agent For open-ended questions - *"Are customers on the Pro plan churning more than last quarter, and why?"*:

- **Decomposes** the question into sub-questions
- **Investigates each in parallel**, from multiple sources: ticket history (vector search), structured DB aggregation, KB docs
- **Synthesizes** findings into a report with **citations to every source used**
- **States its own confidence** and explicitly names what it couldn't determine - "I found X but couldn't verify Y" is a correct and valuable answer; a confident fabrication is the failure mode this guards against
- Long-running: streams progress ("Investigating 3 of 5 sub-questions..."), and must survive being closed and reopened

Voice Channel Expose the agent over voice, reusing the same core (not a parallel implementation):

- Browser mic capture → STT → agent → TTS → playback
- **Streaming both directions** - waiting for a full response before speaking makes it feel broken. Target sub-second time-to-first-audio
- **Voice-specific design problems you must solve:**
 - Responses must be rewritten for speech - a formatted table or bulleted list is unspeakable. Same agent core, different output formatter
 - Tool execution takes seconds; silence feels like a dropped call. Design the filler/progress behavior
 - **HITL over voice:** how does an approval-required action work when the user is on a call? This has no obvious answer - design one and defend it
 - Barge-in: the user interrupts mid-response. Handle it

Frontend additions

- Multi-agent visualizer: which agent handled what, with handoffs on a timeline
- Research report view with expandable citations and per-finding confidence
- Voice interface: waveform, live transcript, agent-activity indicator during tool calls
- Comparison dashboard: single-agent vs multi-agent on latency, tokens, and success rate (your evidence for the written analysis)

Backend Requirements

- **Agent registry** - adding a specialist is configuration, not surgery on the coordinator
- **Shared memory across agents:** what does a specialist see from earlier in the conversation? Design it explicitly
- **Cost tracking per agent** - multi-agent multiplies LLM calls; you must be able to show where tokens go
- **Voice:** WebSocket, JWT in the `Sec-WebSocket-Protocol` header (not the URL - credentials in URLs land in access logs), TTS response caching for repeated phrases

Evaluation Criteria

Criteria	Marks
Multi-Agent Architecture (isolation, handoff contract, loop prevention)	20
Coordinator Routing + Multi-Agent Request Merging	15
Research Agent (decomposition, parallel investigation, citations, confidence)	20
Voice Channel (streaming, latency, voice-specific handling)	20
Multi-Agent vs Single-Agent Written Analysis (with measured data)	15
Cost Tracking & Comparison Dashboard	10
Total	100

Important Instructions

- Voice must reuse the Build 1/2 agent core - a separate voice-only implementation defeats the purpose
- The multi-agent analysis must include **real measured numbers** from your comparison dashboard, not assertions
- A research agent that never says "I couldn't determine this" is fabricating; demonstrate a case where it correctly reports uncertainty

Final Submission

1. GitHub repo - single repo, three tagged releases (build-1, build-2, build-3)
2. README - architecture, setup, demo instructions
3. docs/
 - ├── agent-loop.md (Build 1 - loop design, caps, error handling)
 - ├── tool-design.md (Build 1 - schemas, validation, safety rails)
 - ├── planning-reflection.md (Build 2 - criteria, thresholds, revision caps)
 - ├── hitl-design.md (Build 2 - approval tiers, pause/resume, timeouts)
 - ├── routing-strategy.md (Build 2 - three paths + traffic distribution data)
 - ├── multi-agent-analysis.md (Build 3 - vs single agent, with measured numbers)
 - ├── voice-design.md (Build 3 - latency, formatting, HITL over voice)
 - └── architecture-diagrams/ (one per build - system evolution)
4. Demo video covering the required failure/edge demonstrations
5. Cost report: tokens and \$ per request type across all three builds

Required Demonstrations (the review focuses on these, not the happy path)

- | # | Demonstration |
|----|--|
| 1 | Agent hits the iteration cap and terminates gracefully |
| 2 | Out-of-policy refund rejected at the code level , not by prompt |
| 3 | Tool returns an error; agent recovers and continues |
| 4 | Plan revised mid-execution after an unexpected tool result |
| 5 | Reflection catches a bad draft and the agent fixes it |
| 6 | Approval pause → server restart → approve → correct resumption |
| 7 | Approval rejected → agent handles it without crashing |
| 8 | A request routed to deterministic workflow instead of the agent |
| 9 | Multi-agent request touching two specialists, merged coherently |
| 10 | Research agent reporting uncertainty on something it couldn't verify |
| 11 | Voice conversation including a tool call, with the silence problem handled |

Review Format

Per build:

- Live demo of that build's required demonstrations - failure paths, not happy paths
- 2-3 design decisions challenged; defend or revise openly
- One cost question per build ("this ran 6 LLM calls - which were necessary?")
- Build 3 review is a full architecture review: present the multi-agent analysis and defend whether the added complexity earned its cost

What You'll Actually Learn Here

1. Prompts are not a security boundary. Code-enforced limits are. This is the lesson that separates production agent systems from demos, and Build 1 makes you feel it.

2. The failure paths are the system. A happy-path agent demo takes an afternoon. Iteration caps, tool errors, rejected approvals, resumption after restart, delegation loops - that's the actual engineering, and it's most of the grade here.

3. Autonomy is a dial, not a switch. The routing layer in Build 2 will likely show most traffic doesn't need an autonomous agent at all. Knowing when *not* to use an agent is a senior judgment, and you'll have measured data to back it.

4. Statefulness is the hard part. An agent that pauses for approval and resumes hours later on a different instance is a distributed systems problem wearing an AI costume. Your Phase 1 backend depth is exactly what makes it solvable.

5. Multi-agent is frequently over-applied. Build 3 asks you to measure it rather than assume it. Whichever way your data lands, you'll be able to argue it with evidence - which is rarer, and more valuable, than having built one.